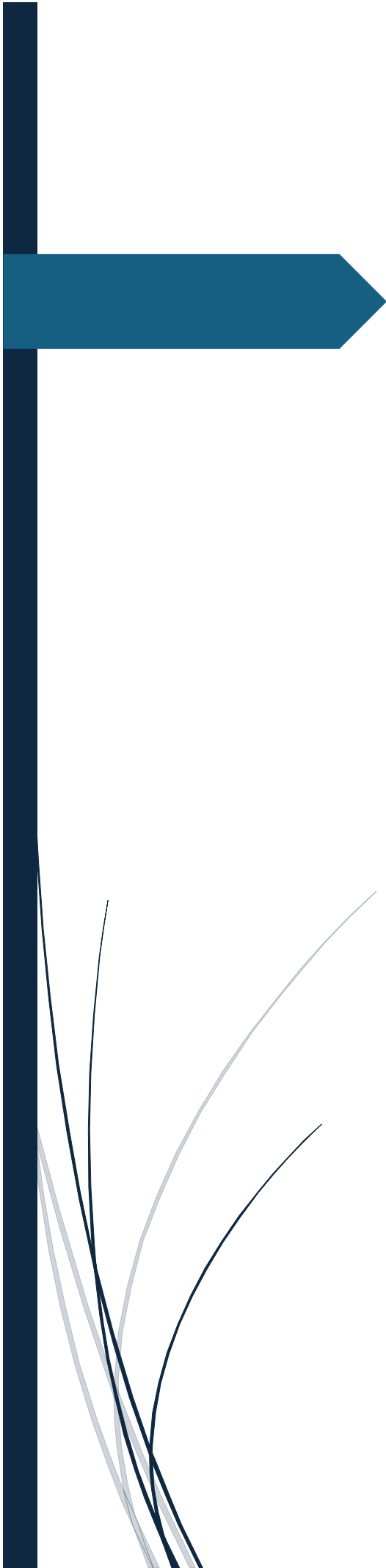




MILESTONE 5 - SEN371

Scentigue - online perfume store

Group Members:

Tanya Richards (603087)

Mickayla Combrink (602923)

Momelezi Simnikiwe Tyini (602059)

Nkosinathi Mathenjwa (577480)

Lecturer:

Reason Sithole

Contents

1. Introduction and Scope.....	2
2. Test Strategy.....	2
2.1 Pyramid Rationale	3
3. Test-Driven Development.....	3
3.1 catchAsync and Synchronous Throws	3
3.2 Administrator ordering listing	3
4. Results	4
4.1 Summary	4
4.2 Backend - 11 suites, 134 tests.....	4
4.3 Client - 7 files, 94 tests.....	4
4.4 End-to-End - 14 tests	5
5. Coverage Analysis.....	5
5.1 Backend	5
5.2 Client	5
5.3 Thresholds Enforced.....	6
6. Defects Found by Testing	6
7. User Acceptance Testing.....	7
7.1 Tasks	7
7.2 Severity Scale.....	8
8. Test Maintenance	8
9. Reproducing Results	8
10. Known Limitations.....	9

1. Introduction and Scope

This report covers testing of Scentigue, an online perfume store built with an Express/MongoDB API and a React client. It outlines the testing approach, responsibilities of each layer, findings, and reproducibility of results.

In scope: API, React client, and customer/admin journeys.

Out of scope: MongoDB Atlas, third-party libraries, and real payment processing.

All 242 automated tests pass. Coverage thresholds are enforced by test runners, failing the build if breached.

2. Test Strategy

Testing is structured into five layers, each answering a distinct question. Behaviours are tested at the lowest layer that can prove them, avoiding duplication higher up.

Layer	Question	Tool	Location
Unit	Does this function work in isolation?	Jest/Vitest	tests/*.unit.test.js, client/src/**/*.test.jsx
Component	Does this module behave with collaborators?	Vitest + Testing Library	client/src/**/*.test.jsx
Integration	Does an HTTP request yield the right DB effect/response?	Jest + Supertest + mongodb-memory-server	tests/*.integration.test.js
Function (E2E)	Can a user complete the journey in a browser?	Playwright	client/tests/e2e/*.spec.js
User (UAT)	Can a real person complete the journey unaided?	Scripted sessions	docs/UAT_SCRIPT.md

2.1 Pyramid Rationale

Unit and integration tests form the bulk: they are fast, precise, and pinpoint failing functions. End-to-end tests are deliberately few, slower and more brittle, reserved for high-risk journeys where cross-layer integration matters (e.g., checkout, where payment and stock checks must align before writing an order).

3. Test-Driven Development

Two milestone changes were written test-first, with failing runs kept as evidence that the tests genuinely exercised the defects before fixes.

3.1 `catchAsync` and Synchronous Throws

A unit test asserted that a route handler throwing synchronously should still reach Express via `next()`. It failed:

- `catchAsync` › a synchronous throw is caught too
Expected number of calls: 1
Received number of calls: 0

The handler was invoked before the promise chain, so the throw escaped and could crash the process. The fix moved the call inside `.then()`, and the same test passed unchanged:

- ```
const catchAsync = (fn) => (req, res, next) =>
 Promise.resolve()
 .then(() => fn(req, res, next))
 .catch(next);
```

### 3.2 Administrator ordering listing

GET `/api/orders/all` was specified entirely in tests before implementation:

- Admin sees all orders
- Customer receives 403
- GET `/api/orders` still returns only the caller's orders
- `all` is not parsed as an order ID

The route, controller, service, and repository were then written until all four tests passed.

## 4. Results

### 4.1 Summary

| Test Group                | Count |
|---------------------------|-------|
| Backend unit tests        | 25    |
| Backend integration tests | 109   |
| Client unit tests         | 59    |
| Client component tests    | 35    |
| End-to-end function tests | 14    |
| Total                     | 242   |

### 4.2 Backend - 11 suites, 134 tests

| Suite                             | Tests | Type        |
|-----------------------------------|-------|-------------|
| appError.unit.test.js             | 9     | Unit        |
| catchAsync.unit.test.js           | 3     | Unit        |
| orderService.unit.test.js         | 13    | Unit        |
| auth.integration.test.js          | 12    | Integration |
| cart.integration.test.js          | 10    | Integration |
| category.integration.test.js      | 14    | Integration |
| errorHandling.integration.test.js | 6     | Integration |
| order.integration.test.js         | 15    | Integration |
| product.integration.test.js       | 32    | Integration |
| refreshToken.integration.test.js  | 12    | Integration |
| repositories.integration.test.js  | 8     | Integration |

### 4.3 Client - 7 files, 94 tests

| Suite                           | Test | Type      |
|---------------------------------|------|-----------|
| utils/payment.test.js           | 32   | Unit      |
| utils/format.test.js            | 13   | Unit      |
| api/client.test.js              | 24   | Unit      |
| components/ProductCard.test.jsx | 8    | Component |

|                                         |   |           |
|-----------------------------------------|---|-----------|
| components/ProductImage.test.jsx        | 5 | Component |
| components/admin/ConfirmDialog.test.jsx | 8 | Component |
| routes/AdminRoute.test.jsx              | 4 | Component |

## 4.4 End-to-End - 14 tests

| Specification    | Tests | Journey       |
|------------------|-------|---------------|
| checkout.spec.js | 9     | Customer      |
| admin.spec.js    | 5     | Administrator |

Playwright drives the built bundle in Chromium. The network is stubbed for deterministic journeys, requiring no running backend, while application code, routing, and rendering remain real.

# 5. Coverage Analysis

## 5.1 Backend

| Metric    | Functions | Branches | Lines  |
|-----------|-----------|----------|--------|
| Measured  | 91.60%    | 73.78%   | 92.99% |
| Threshold | 80%       | 70%      | 80%    |

Models, routes, and controllers are near 100%. Gaps remain in config/db.js (tests use in-memory DB) and error branches in errorHandler.js. coverage excludes src/api/\*\* and src/utls/errorMessages.js (dead code), which previously lowered backend figures by ~16 points.

## 5.2 Client

| File            | Stmts  | Branch | Funcs  | Lines  |
|-----------------|--------|--------|--------|--------|
| All files       | 19.15  | 17.75  | 17.52  | 18.90  |
| api/client.js   | 98.33  | 100.00 | 96.00  | 98.18  |
| utls/payment.js | 100.00 | 100.00 | 100.00 | 100.00 |
| utls/format.js  | 100.00 | 90.00  | 100.00 | 100.00 |

|                   |        |       |        |        |
|-------------------|--------|-------|--------|--------|
| ProductCard.jsx   | 100.00 | 93.75 | 100.00 | 100.00 |
| ProductImage.jsx  | 100.00 | 90.00 | 100.00 | 100.00 |
| ConfirmDialog.jsx | 93.75  | 84.21 | 100.00 | 100.00 |
| pages/*           | 0.00   | 0.00  | 0.00   | 0.00   |

Headline coverage is low by design. Logic modules (payment validation, delivery rules, route guards, etc.) are near 100%. Page components are markup-heavy and instead are covered by 14 Playwright journeys, which validate real user outcomes (e.g., order creation).

### 5.3 Thresholds Enforced

| Scope            | Stmts | Branch | Funcs | Lines |
|------------------|-------|--------|-------|-------|
| Backend, global  | 80    | 70     | 80    | 80    |
| Client, global   | 17    | 16     | 15    | 17    |
| utils/payment.js | 95    | 90     | 90    | 95    |
| utils/format.js  | 90    | 85     | 85    | 90    |
| api/client.js    | 85    | 75     | 85    | 85    |
| ProductCard.jsx  | 95    | 85     | 95    | 95    |
| ProductImage.jsx | 95    | 85     | 95    | 95    |

Breaches fail the run, preventing silent decay. HTML reports are generated in coverage/ for both backend and client.

## 6. Defects Found by Testing

Four defects were identified in this milestone, all fixed and covered by regression tests.

| # | Defect                                                                                       | Found By  | Severity |
|---|----------------------------------------------------------------------------------------------|-----------|----------|
| 1 | catchAsync failed to catch synchronous throws, crashing the process instead of returning 500 | Unit test | Major    |
| 2 | Confirm dialog buttons unclickable in browser (Bootstrap pointer-events issue)               | E2E test  | Critical |

|   |                                                                                 |          |          |
|---|---------------------------------------------------------------------------------|----------|----------|
| 3 | /admin declared twice; unguarded route allowed sign-out access to admin console | E2E test | Critical |
| 4 | Admin panel displayed hardcoded figures, no endpoint calls                      | E2E test | Major    |

#### Notes:

- Defect 2 highlights the need for multiple layers: component tests passed in jsdom, but only a real browser exposed the issue.
- Defect 3 was the most serious, as duplicate router declarations left the admin console accessible without authentication.

## 7. User Acceptance Testing

Protocols, task wording, and observation sheets are in docs/UAT\_SCRIPT.md. Sessions involve 5-8 new participants; five typically surface most usability issues, while more than eight rarely add value.

Participants think aloud; observers do not guide or answer navigation questions. Help is given only when a participant is fully blocked, and that task is then marked as failed.

### 7.1 Tasks

| # | Task                                          | Success Criterion                              |
|---|-----------------------------------------------|------------------------------------------------|
| 1 | Find an oud fragrance under R900              | Reaches product detail pag unaided             |
| 2 | Set up to place an order                      | Account created, sign in                       |
| 3 | Add two items, then reduce to one             | Cart shows one unit                            |
| 4 | Buy with card 4242 4242 4242 4242             | Confirmation page reached, order number stated |
| 5 | Attempt payment with card 4000 0000 0000 0000 | Recognise failure, explains next step          |
| 6 | Check orders and delivery                     | Order history reached, delivery window stated  |
| 7 | Admin: restock 15 units of sold-out fragrance | Stock shows 15 in admin list                   |
| 8 | Admin: mark order as sent and paid            | Order shows Shipping + Paid                    |

## 7.2 Severity Scale

| Level | Meaning                        | Action                |
|-------|--------------------------------|-----------------------|
| 1     | Cosmetic - noticed, no impact  | Fix if time allows    |
| 2     | Minor - slowed participant     | Fix before submission |
| 3     | Major - needed help/wrong path | Must fix              |
| 4     | Critical - task incomplete     | Must fix, then reset  |

## 8. Test Maintenance

Nine redundant tests were removed. `errorHandling.integration.test.js` has duplicated the category CRUD suite (create, duplicate, update, delete, not-found), meaning failures appeared twice and wasted runtime.

The suite now covers only the four centralised error handlers plus 404 and health checks, with each behaviour asserted exactly once.

### Removed Duplicates:

- Create new list → 201
- Missing required description → 400
- Update a category → 200
- Delete a category → 204
- Delete non-existent category → 404
- Get non-existent but valid ID → 404 (duplicate of product suite)
- Unknown /api route → 404 (identical to test above)
- Malformed ID → 400 (kept once in handler suite)

## 9. Reproducing Results

### Backend (from project root):

- `npm install`

```
npm run test:coverage
```

**Client (from client/):**

- ```
cd client  
npm install  
npm run test:coverage  
npx playwright install chromium    # first run only  
npm run test:e2e
```

Notes:

- Backend integration suite downloads a MongoDB binary on first run (requires internet).
- Server uses Jest, client uses Vitest ; each must be run from its own folder.
- Jest ignores client/ to avoid parsing Vitest/Playwright specs (JSX/ESM syntax).

10. Known Limitations

- Payment is simulated - no gateway is contacted, no card charged. Tests assert simulation rules only.
- Delivery fees are presentational; the API charges for items only, so no server-side fee assertions.
- Coverage of presentational pages is intentionally low; they are exercised by end-to-end journeys.
- Load and security testing were not performed; both are out of scope for this project.